

SQL ASSIGNMENT 1

1. Create Database e_commerce

⇒ **Create Database e_commerce;**

use e_commerce;

show databases;

	Database
▶	e_commerce
	information_schema
	mysql
	performance_schema
	sys

2. Create following Tables:

Customers:

- a. customer_id - int auto-increment primary key
- b. name - varchar(50)
- c. email - varchar(50)
- d. mobile - varchar(15)

Products:

- a. id - int
- b. name - varchar(50) not null
- c. description - varchar(200)
- d. price - decimal(10, 2) not null
- e. category - varchar(50)

⇒ **Customers:**

```
CREATE TABLE Customers (  
customer_id INT AUTO_INCREMENT PRIMARY KEY,  
name VARCHAR(50),  
email VARCHAR(50),  
mobile VARCHAR(15)  
);
```

	Field	Type	Null	Key	Default	Extra
▶	customer_id	int	NO	PRI	NULL	auto_increment
	name	varchar(50)	YES		NULL	
	email	varchar(50)	YES		NULL	
	mobile	varchar(15)	YES		NULL	

⇒ **Products:**

```
CREATE TABLE Products (  
id INT,  
name VARCHAR(50) NOT NULL,  
description VARCHAR(200),  
price DECIMAL(10,2) NOT NULL,  
category VARCHAR(50)  
);
```

	Field	Type	Null	Key	Default	Extra
▶	id	int	YES		NULL	
	name	varchar(50)	NO		NULL	
	description	varchar(200)	YES		NULL	
	price	decimal(10,2)	NO		NULL	
	category	varchar(50)	YES		NULL	

3. Modify Tables(using Alter keyword):

a. Add not null on name and email in the Customers table

⇒ ALTER TABLE Customers MODIFY name VARCHAR(50) NOT NULL, MODIFY email VARCHAR(50) NOT NULL;

	Field	Type	Null	Key	Default	Extra
▶	customer_id	int	NO	PRI	NULL	auto_increment
	name	varchar(50)	NO		NULL	
	email	varchar(50)	NO		NULL	
	mobile	varchar(15)	YES		NULL	

b. Add unique key on email in the Customers table:

⇒ ALTER TABLE Customers ADD CONSTRAINT u_email UNIQUE (email);

	Field	Type	Null	Key	Default	Extra
▶	customer_id	int	NO	PRI	NULL	auto_increment
	name	varchar(50)	NO		NULL	
	email	varchar(50)	NO	UNI	NULL	
	mobile	varchar(15)	YES		NULL	

c. Add column age in the Customers table:

⇒ ALTER TABLE Customers ADD COLUMN age INT;

	Field	Type	Null	Key	Default	Extra
▶	customer_id	int	NO	PRI	NULL	auto_increment
	name	varchar(50)	NO		NULL	
	email	varchar(50)	NO	UNI	NULL	
	mobile	varchar(15)	YES		NULL	
	age	int	YES		NULL	

d. Change column name from id to product_id in the Products table;

=> ALTER TABLE Products CHANGE COLUMN id product_id INT;

	Field	Type	Null	Key	Default	Extra
▶	product_id	int	YES		NULL	
	name	varchar(50)	NO		NULL	
	description	varchar(200)	YES		NULL	
	price	decimal(10,2)	NO		NULL	
	category	varchar(50)	YES		NULL	

e. Add primary key and auto increment on product_id in the Products table

=> ALTER TABLE Products MODIFY COLUMN product_id INT AUTO_INCREMENT PRIMARY KEY;

	Field	Type	Null	Key	Default	Extra
▶	product_id	int	NO	PRI	NULL	auto_increment
	name	varchar(50)	NO		NULL	
	description	varchar(200)	YES		NULL	
	price	decimal(10,2)	NO		NULL	
	category	varchar(50)	YES		NULL	

f. Change datatype of description from varchar to text in the Products table:

=> ALTER TABLE Products MODIFY COLUMN description text;

	Field	Type	Null	Key	Default	Extra
▶	product_id	int	NO	PRI	NULL	auto_increment
	name	varchar(50)	NO		NULL	
	description	text	YES		NULL	
	price	decimal(10,2)	NO		NULL	
	category	varchar(50)	YES		NULL	

4. Create table Order:

- order_id - int auto-increment primary key
- customer_id - int -foreign key
- product_id - int
- quantity - int not null,
- order_date - date not null,
- status - enum(Pending, Success, Cancel),
- payment_method - enum(Credit, Debit, UPI),
- total_amount - decimal(10, 2) not null

```
⇒ CREATE TABLE Orders (
    order_id INT AUTO_INCREMENT PRIMARY KEY,
    customer_id INT,
    product_id INT,
    quantity INT NOT NULL,
    order_date DATE NOT NULL,
    status enum('Pending', 'Success', 'Cancel'),
    payment_method enum('Credit', 'Debit', 'UPI'),
    total_amount decimal(10,2) NOT NULL
);
```

	Field	Type	Null	Key	Default	Extra
►	order_id	int	NO	PRI	NULL	auto_increment
	customer_id	int	YES		NULL	
	product_id	int	YES		NULL	
	quantity	int	NO		NULL	
	order_date	date	NO		NULL	
	status	enum('Pending', 'Success', 'Cancel')	YES		NULL	
	payment_method	enum('Credit', 'Debit', 'UPI')	YES		NULL	
	total_amount	decimal(10,2)	NO		NULL	

5. Modify Orders Table(using Alter keyword):

a. Change table name Order -> Orders

```
⇒ ALTER TABLE Order RENAME TO Orders;
```

b. Set default value pending in status.

```
⇒ ALTER TABLE Orders MODIFY COLUMN status ENUM('Pending', 'Success', 'Cancel') DEFAULT 'Pending';
```

	Field	Type	Null	Key	Default	Extra
►	order_id	int	NO	PRI	NULL	auto_increment
	customer_id	int	YES		NULL	
	product_id	int	YES		NULL	
	quantity	int	NO		NULL	
	order_date	date	NO		NULL	
	status	enum('Pending', 'Success', 'Cancel')	YES		Pending	
	payment_method	enum('Credit', 'Debit', 'UPI')	YES		NULL	
	total_amount	decimal(10,2)	NO		NULL	

c. Modify payment_method ENUM to add one more value: 'COD'

```
⇒ ALTER TABLE Orders MODIFY COLUMN payment_method ENUM('Credit', 'Debit', 'UPI', 'COD');
```

	Field	Type	Null	Key	Default	Extra
►	order_id	int	NO	PRI	NULL	auto_increment
	customer_id	int	YES		NULL	
	product_id	int	YES		NULL	
	quantity	int	NO		NULL	
	order_date	date	NO		NULL	
	status	enum('Pending','Success','Cancel')	YES		Pending	
	payment_method	enum('Credit','Debit','UPI','COD')	YES		NULL	
	total_amount	decimal(10,2)	NO		NULL	

d. Make product id as foreign key

⇒ ALTER TABLE Orders ADD CONSTRAINT fk_orders_product FOREIGN KEY (product_id) REFERENCES Products(product_id);

	Field	Type	Null	Key	Default	Extra
►	order_id	int	NO	PRI	NULL	auto_increment
	customer_id	int	YES		NULL	
	product_id	int	YES	MUL	NULL	
	quantity	int	NO		NULL	
	order_date	date	NO		NULL	
	status	enum('Pending','Success','Cancel')	YES		Pending	
	payment_method	enum('Credit','Debit','UPI','COD')	YES		NULL	
	total_amount	decimal(10,2)	NO		NULL	

6. Insert 20 sample records in all the tables.

⇒

	customer_id	name	email	mobile	age
►	1	Rohan Singh	rohan@example.com	9876543212	28
	2	Aarav Jain	aarav@example.com	9876543213	22
	3	Kabir Rao	kabir@example.com	9876543214	35
	4	Siddharth	siddharth@example.com	9876543215	40
	5	Arjun	arjun@example.com	9876543216	29
	6	Yash	yash@example.com	9876543217	20
	7	Vivek	vivek@example.com	9876543218	38
	8	Rahul	rahul@example.com	9876543219	32
	9	Amit	amit@example.com	9876543220	45
	10	Gaurav	gaurav@example.com	9876543221	31
	11	Saurabh	saurabh@example.com	9876543222	42
	12	Manish	manish@example.com	9876543223	36
	13	Himanshu	himanshu@example.com	9876543224	39
	14	Rishabh	rishabh@example.com	9876543225	33
	15	Siddharth	siddharth2@example.com	9876543226	41
	16	Dhruv	dhruv@example.com	9876543227	26
	17	Shrey	shrey@example.com	9876543228	24
	18	Kunal	kunal@example.com	9876543229	34
	19	Mayank	mayank@example.com	9876543230	43
	20	Sohan	sohan@example.com	9876543231	27
✱	NULL	NULL	NULL	NULL	NULL

	product_id	name	description	price	category
▶	1	TV	4K Smart TV	800.00	Electronics
	2	Tablet	Android Tablet	250.00	Electronics
	3	Headphones	Wireless Headphones	150.00	Electronics
	4	Smartwatch	Android Smartwatch	200.00	Electronics
	5	Gaming Console	PlayStation	500.00	Electronics
	6	Printer	Wireless Printer	100.00	Office
	7	Scanner	Document Scanner	80.00	Office
	8	Notebook	College Notebook	50.00	Stationery
	9	Pen Drive	32GB Pen Drive	20.00	Computer Accessories
	10	Keyboard	Wireless Keyboard	30.00	Computer Accessories
	11	Mouse	Wireless Mouse	25.00	Computer Accessories
	12	Camera	DSLR Camera	1200.00	Electronics
	13	Speaker	Bluetooth Speaker	80.00	Electronics
	14	Power Bank	Portable Power Bank	40.00	Electronics
	15	Book	Textbook	100.00	Stationery
	16	Chair	Office Chair	150.00	Furniture
	17	Desk	Office Desk	200.00	Furniture
	18	Shirt	Casual Shirt	80.00	Clothing
	19	Pants	Jeans Pants	100.00	Clothing
	20	Shoes	Running Shoes	120.00	Clothing
▲	NULL	NULL	NULL	NULL	NULL

	order_id	customer_id	product_id	quantity	order_date	status	payment_method	total_amount
▶	21	1	1	2	2024-06-15	Success	Credit	900.00
	22	2	2	1	2024-06-16	Pending	UPI	300.00
	23	1	3	1	2024-06-17	Success	Credit	150.00
	24	3	4	2	2024-06-19	Success	Debit	400.00
	25	1	5	1	2024-06-20	Success	Credit	500.00
	26	2	6	1	2024-06-21	Pending	UPI	100.00
	27	2	7	2	2024-06-22	Success	Credit	200.00
	28	4	8	1	2024-06-23	Success	Debit	80.00
	29	1	9	3	2024-06-24	Pending	UPI	450.00
	30	5	10	1	2024-06-25	Success	Credit	30.00
	31	6	11	2	2024-06-26	Success	Credit	50.00
	32	7	12	1	2024-06-27	Pending	UPI	1200.00
	33	8	13	1	2024-06-28	Success	Credit	80.00
	34	9	14	2	2024-06-29	Success	Debit	40.00
	35	10	15	1	2024-06-30	Pending	UPI	150.00
	36	11	16	2	2024-07-01	Success	Credit	200.00
	37	12	17	1	2024-07-02	Success	Credit	120.00
	38	13	18	3	2024-07-03	Success	Credit	300.00
	39	14	19	1	2024-07-04	Pending	UPI	250.00
	40	15	20	2	2024-07-05	Success	Debit	350.00
◀	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

7. Perform following queries:

a. Count the number of products as product_count in each category.

⇒ SELECT category, COUNT(*) AS product_count FROM Products GROUP BY category;

	category	product_count
▶	Electronics	8
	Office	2
	Stationery	2
	Computer Accessories	3
	Furniture	2
	Clothing	3

b. Retrieve all products that belong to the 'Electronics' category, have a price between \$50 and \$500, and whose name contains the letter 'a'.

⇒ SELECT * FROM Products WHERE category = 'Electronics' AND price BETWEEN 50 AND 500 AND name LIKE '%a%';

	product_id	name	description	price	category
▶	2	Tablet	Android Tablet	250.00	Electronics
	3	Headphones	Wireless Headphones	150.00	Electronics
	4	Smartwatch	Android Smartwatch	200.00	Electronics
	5	Gaming Console	PlayStation	500.00	Electronics
	13	Speaker	Bluetooth Speaker	80.00	Electronics
•	NULL	NULL	NULL	NULL	NULL

c. Get the top 5 most expensive products in the 'Electronics' category, skipping the first 2.

⇒ SELECT * FROM Products WHERE category = 'Electronics' ORDER BY price DESC LIMIT 5 OFFSET 2;

	product_id	name	description	price	category
▶	5	Gaming Console	PlayStation	500.00	Electronics
	2	Tablet	Android Tablet	250.00	Electronics
	4	Smartwatch	Android Smartwatch	200.00	Electronics
	3	Headphones	Wireless Headphones	150.00	Electronics
	13	Speaker	Bluetooth Speaker	80.00	Electronics
•	NULL	NULL	NULL	NULL	NULL

d. Retrieve customers who have not placed any orders.

⇒ SELECT * FROM Customers WHERE customer_id NOT IN (SELECT DISTINCT customer_id FROM Orders);

	customer_id	name	email	mobile	age
▶	16	Dhruv	dhruv@example.com	9876543227	26
	17	Shrey	shrey@example.com	9876543228	24
	18	Kunal	kunal@example.com	9876543229	34
	19	Mayank	mayank@example.com	9876543230	43
	20	Sohan	sohan@example.com	9876543231	27
•	NULL	NULL	NULL	NULL	NULL

e. Find the average total amount spent by each customer.

⇒ SELECT customer_id, AVG(total_amount) AS avg_spent FROM Orders GROUP BY customer_id;

	customer_id	avg_spent
▶	1	500.000000
	2	200.000000
	3	400.000000
	4	80.000000
	5	30.000000
	6	50.000000
	7	1200.000000
	8	80.000000
	9	40.000000
	10	150.000000
	11	200.000000
	12	120.000000
	13	300.000000
	14	250.000000
	15	350.000000

f. Get the products that have a price less than the average price of all products.

⇒ SELECT * FROM Products WHERE price < (SELECT AVG(price) FROM Products);

	product_id	name	description	price	category
▶	3	Headphones	Wireless Headphones	150.00	Electronics
	4	Smartwatch	Android Smartwatch	200.00	Electronics
	6	Printer	Wireless Printer	100.00	Office
	7	Scanner	Document Scanner	80.00	Office
	8	Notebook	College Notebook	50.00	Stationery
	9	Pen Drive	32GB Pen Drive	20.00	Computer Accessories
	10	Keyboard	Wireless Keyboard	30.00	Computer Accessories
	11	Mouse	Wireless Mouse	25.00	Computer Accessories
	13	Speaker	Bluetooth Speaker	80.00	Electronics
	14	Power Bank	Portable Power Bank	40.00	Electronics
	15	Book	Textbook	100.00	Stationery
	16	Chair	Office Chair	150.00	Furniture
	17	Desk	Office Desk	200.00	Furniture
	18	Shirt	Casual Shirt	80.00	Clothing
	19	Pants	Jeans Pants	100.00	Clothing
	20	Shoes	Running Shoes	120.00	Clothing
	NULL	NULL	NULL	NULL	NULL

g. Calculate the total quantity of products ordered by each customer:

⇒ SELECT customer_id, SUM(quantity) AS total_quantity FROM Orders
GROUP BY customer_id;

	customer_id	total_quantity
▶	1	7
	2	4
	3	2
	4	1
	5	1
	6	2
	7	1
	8	1
	9	2
	10	1
	11	2
	12	1
	13	3
	14	1
	15	2

h. List all orders along with customer name and product name.

⇒ SELECT Orders.order_id, Customers.name AS customer_name, Products.name AS product_name FROM Orders

INNER JOIN Customers ON Orders.customer_id = Customers.customer_id

INNER JOIN Products ON Orders.product_id = Products.product_id;

	order_id	customer_name	product_name
▶	21	Rohan Singh	TV
	22	Aarav Jain	Tablet
	23	Rohan Singh	Headphones
	24	Kabir Rao	Smartwatch
	25	Rohan Singh	Gaming Console
	26	Aarav Jain	Printer
	27	Aarav Jain	Scanner
	28	Siddharth	Notebook
	29	Rohan Singh	Pen Drive
	30	Arjun	Keyboard
	31	Yash	Mouse
	32	Vivek	Camera
	33	Rahul	Speaker
	34	Amit	Power Bank
	35	Gaurav	Book
	36	Saurabh	Chair
	37	Manish	Desk
	38	Himanshu	Shirt
	39	Rishabh	Pants
	40	Siddharth	Shoes

i. Find products that have never been ordered.

⇒ SELECT * FROM Products WHERE product_id NOT IN (SELECT DISTINCT product_id FROM Orders);

	product_id	name	description	price	category
•	NULL	NULL	NULL	NULL	NULL